ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

Département Informatique

Rapport de Projet

Algorithmes de recherche heuristiquement Ordonnée+ Chaîne de Markov

Réalisé par :
Anass MAKHLOUK

Encadré par :
Prof. MESTARI

Master : SDIA
Module : Bases de l'intelligence artificielle

Année Universitaire : 2025 – 2026

Table des matières

1 Introduction **2**

2 Modélisation mathématique **2**
2.1 Espace d'états et graphe de recherche 2
2.2 Algorithmes de recherche 2
2.3 Chaîne de Markov et incertitude d'action 3
2.4 Analyse d'absorption 3

3 Implémentation **4**
3.1 Recherche heuristique 4
3.2 Construction de la matrice de transition 5
3.3 Conception des grilles 5

4 Expériences et résultats **6**
4.1 E.1 : Comparaison UCS vs Greedy vs A^* 6
4.2 E.2 : Impact de l'incertitude ε sur la robustesse 9
4.3 E.3 : Comparaison de deux heuristiques admissibles 11
4.4 E.4 : Weighted A^* : compromis vitesse vs optimalité 12
4.5 Validation Monte-Carlo 13

5 Discussion **14**
5.1 Intérêt et garanties des heuristiques 14
5.2 Impact de l'incertitude sur un plan déterministe 14
5.3 Chemin optimal prévu vs performance probabiliste réelle 14

6 Conclusion et perspectives **14**

1 Introduction

La planification de chemin en présence d'incertitude constitue un problème fondamental en intelligence artificielle, avec des applications directes en robotique mobile, logistique et navigation autonome. Dans un cadre déterministe, les algorithmes de recherche heuristique — notamment A^* — permettent de calculer un chemin de coût minimal en exploitant une estimation admissible du coût restant. Cependant, l'exécution réelle d'un plan est soumise à des perturbations : glissement, erreur d'actionneurs, conditions environnementales variables. La question centrale est alors : *dans quelle mesure un chemin optimal déterministe demeure-t-il performant lorsque les transitions sont stochastiques ?*

Ce projet propose une approche hybride en deux phases :

1. **Planification déterministe** par recherche heuristiquement ordonnée (A^* et variantes) sur une grille 2D avec obstacles, en comparant UCS, Greedy Best-First et A^*.
2. **Analyse stochastique** par modélisation de la dynamique d'exécution sous forme de chaîne de Markov à temps discret, avec états absorbants (GOAL, FAIL), calcul matriciel exact et validation par simulation Monte-Carlo.

Le rapport est organisé comme suit : la section 2 décrit le modèle mathématique, la section 3 présente l'implémentation, la section 4 détaille les quatre expériences, et la section 6 conclut avec les enseignements et perspectives.

2 Modélisation mathématique

2.1 Espace d'états et graphe de recherche

L'environnement est une grille rectangulaire $\mathcal{G}$ de dimensions $R \times C$. Chaque cellule (r, c) est soit *libre* (état $s \in S$), soit un *obstacle*. L'ensemble des états libres $S \subset \{0, \ldots, R-1\} \times \{0, \ldots, C-1\}$ forme un graphe non orienté sous 4-connexité :

$$\mathcal{N}(r, c) = \big\{(r', c') : |r' - r| + |c' - c| = 1,\ (r', c') \in S\big\}. \tag{1}$$

Le coût de chaque arête est uniforme : $c(s, s') = 1$ pour tout $s' \in \mathcal{N}(s)$. L'agent part d'un état initial s_0 et doit atteindre un état but g.

2.2 Algorithmes de recherche

On considère la famille des algorithmes de type *Best-First Search* utilisant deux structures : OPEN (file de priorité) et CLOSED (ensemble des nœuds déjà développés). À chaque itération, le nœud de OPEN minimisant $f(n)$ est sélectionné et développé.

UCS (Uniform Cost Search). $f(n) = g(n)$, où $g(n)$ est le coût du chemin depuis s_0. UCS est optimal et complet, mais n'utilise aucune information heuristique : il explore de manière uniforme.

Greedy Best-First Search. $f(n) = h(n)$, où $h(n)$ est l'heuristique estimant le coût de n vers g. Greedy est rapide en termes de nœuds développés mais ne garantit pas l'optimalité.

A^*. $f(n) = g(n) + h(n)$. Si h est *admissible* ($h(n) \leq h^*(n)$ pour tout n, où h^* est le coût réel) et *cohérente* ($h(n) \leq c(n, n') + h(n')$ pour tout arc (n, n')), alors A^* est optimal et ne réexpand jamais un nœud [**?**].

Weighted A^*. $f(n) = g(n) + w \cdot h(n)$ avec $w > 1$. La solution trouvée a un coût au plus $w \cdot C^*$ où C^* est le coût optimal. L'intérêt est de réduire le nombre de nœuds développés au prix d'une sous-optimalité bornée.

Heuristique Manhattan. Pour une grille en 4-connexité avec coûts unitaires :

$$h_{\text{Man}}(r, c) = |r - r_g| + |c - c_g| \tag{2}$$

Cette heuristique est admissible et cohérente : elle minore le nombre minimal de déplacements.

2.3 Chaîne de Markov et incertitude d'action

L'exécution du chemin planifié est modélisée par une chaîne de Markov à temps discret $\{X_n\}_{n \geq 0}$ sur l'ensemble d'états augmenté $S' = S \cup \{\textsc{Goal}, \textsc{Fail}\}$.

Politique déterministe. À partir du chemin A^* $(s_0, s_1, \ldots, s_L = g)$, on définit la politique :

$$\pi(s_i) = a_i \quad \text{tel que } s_{i+1} = s_i + \Delta_{a_i}, \quad i = 0, \ldots, L-1. \tag{3}$$

Les états hors du chemin n'ont pas de politique définie et mènent directement à l'état absorbant Fail.

Modèle de transition. Pour un taux de déviation $\varepsilon \in [0, 1)$, lorsque l'agent en état s applique l'action $a = \pi(s)$:

$$P(X_{n+1} = s' \mid X_n = s) = \begin{cases} 1 - \varepsilon & \text{si } s' = \text{résultat}(s, a), \\ \varepsilon/2 & \text{si } s' = \text{résultat}(s, a_\perp^{(1)}), \\ \varepsilon/2 & \text{si } s' = \text{résultat}(s, a_\perp^{(2)}), \end{cases} \tag{4}$$

où $a_\perp^{(1)}$ et $a_\perp^{(2)}$ sont les actions latérales (orthogonales à a). Si le déplacement mène à un obstacle ou hors grille, l'agent reste sur place.

États absorbants. Les états Goal et Fail sont absorbants :

$$P(\textsc{Goal}, \textsc{Goal}) = 1, \qquad P(\textsc{Fail}, \textsc{Fail}) = 1. \tag{5}$$

L'état g (but sur la grille) transitionne vers Goal avec probabilité 1. Tout état hors politique transitionne vers Fail avec probabilité 1.

2.4 Analyse d'absorption

La matrice P admet une décomposition canonique en séparant les n états transitoires et les 2 états absorbants :

$$P = \begin{pmatrix} Q & R \\ \mathbf{0} & I_2 \end{pmatrix}, \tag{6}$$

où $Q \in \mathbb{R}^{n \times n}$ gouverne les transitions entre états transitoires et $R \in \mathbb{R}^{n \times 2}$ les transitions vers les absorbants.

Matrice fondamentale.

$$N = (I_n - Q)^{-1}. \tag{7}$$

L'entrée N_{ij} donne le nombre moyen de visites de l'état j en partant de i avant absorption.

Probabilités d'absorption.

$$B = N \cdot R, \qquad B_{i,0} = \mathbb{P}(\text{absorption en } \textsc{Goal} \mid X_0 = i). \tag{8}$$

Temps moyen d'absorption.

$$\mathbf{t} = N \cdot \mathbf{1}_n, \qquad t_i = \mathbb{E}[\text{temps avant absorption} \mid X_0 = i]. \tag{9}$$

Évolution temporelle. La distribution de probabilité à l'instant n est donnée par :

$$\boldsymbol{\pi}^{(n)} = \boldsymbol{\pi}^{(0)} \cdot P^n, \tag{10}$$

avec $\boldsymbol{\pi}^{(0)}$ la distribution initiale concentrée sur s_0.

3 Implémentation

Le projet est structuré en modules Python :

- `grids.py` : définition des trois grilles (obstacles, positions de départ et but, voisinage 4-connexité).
- `astar.py` : algorithme de recherche générique avec modes UCS, Greedy, A^* et Weighted A^*.
- `markov.py` : construction de P, calcul de $\boldsymbol{\pi}^{(n)}$, analyse d'absorption $(N, B, \mathbf{t})$ et simulation Monte-Carlo.
- `experiments.py` : orchestration des quatre expériences et sérialisation JSON.

3.1 Recherche heuristique

L'algorithme 1 présente la procédure générique.

Algorithm 1 Recherche heuristiquement ordonnée (générique)

Require: Grille $\mathcal{G}$, état initial s_0, but g, heuristique h, mode $\in \{\texttt{ucs}, \texttt{greedy}, \texttt{astar}\}$, poids w

Ensure: Chemin optimal (ou sous-optimal), coût, nœuds développés

```
OPEN ← {(f(s_0), 0, g(s_0), s_0, None)}                ▷ file de priorité (min-heap)
CLOSED ← ∅,   g[s_0] ← 0
while OPEN ≠ ∅ do
    (f, _, g_v, s, parent) ← extraire_min(OPEN)
    if s ∈ CLOSED then continuer
    end if
    CLOSED ← CLOSED ∪ {s}
    if s = g then return reconstituer le chemin via les parents
    end if
    for s' ∈ N(s) \ CLOSED do
        g' ← g_v + c(s, s')
        if g' < g[s'] then
            g[s'] ← g'
            Insérer (f(g', s'), g', s', s) dans OPEN
        end if
    end for
end while
```

Selon le mode : $f = g$ (UCS), $f = h$ (Greedy), $f = g + w \cdot h$ (A^* / Weighted A^*).

3.2 Construction de la matrice de transition

L'algorithme 2 résume la construction de P.

Algorithm 2 Construction de la matrice de transition P

Require: Grille $\mathcal{G}$, politique π, taux de déviation ε, but g

Ensure: Matrice $P \in \mathbb{R}^{(|S|+2)\times(|S|+2)}$

```
P[GOAL, GOAL] ← 1,   P[FAIL, FAIL] ← 1
for chaque état libre s ∈ S do
    if s = g then P[s, GOAL] ← 1 ; continuer
    end if
    if s ∉ dom(π) then P[s, FAIL] ← 1 ; continuer
    end if
    a ← π(s)
    P[s, résultat(s, a)] += 1 − ε
    for a_⊥ ∈ latéraux(a) do
        P[s, résultat(s, a_⊥)] += ε/2
    end for
end for
```

3.3 Conception des grilles

Trois grilles de difficulté croissante ont été conçues avec des propriétés spécifiques pour discriminer les algorithmes :

Table 1 – Caractéristiques des grilles expérimentales.

Grille	Dimensions	Cases libres	Obstacles
Facile	8×8	49	15
Moyenne	12×12	118	26
Difficile	15×15	190	35

Chaque grille comporte deux murs verticaux avec des ouvertures asymétriques (figure 1). Cette topologie crée deux chemins de longueurs distinctes : l'un court (passage par l'ouverture proche) et l'autre long (contournement). Greedy Best-First, guidé exclusivement par la distance au but, est systématiquement attiré vers l'ouverture la plus éloignée, ce qui produit un chemin sous-optimal.

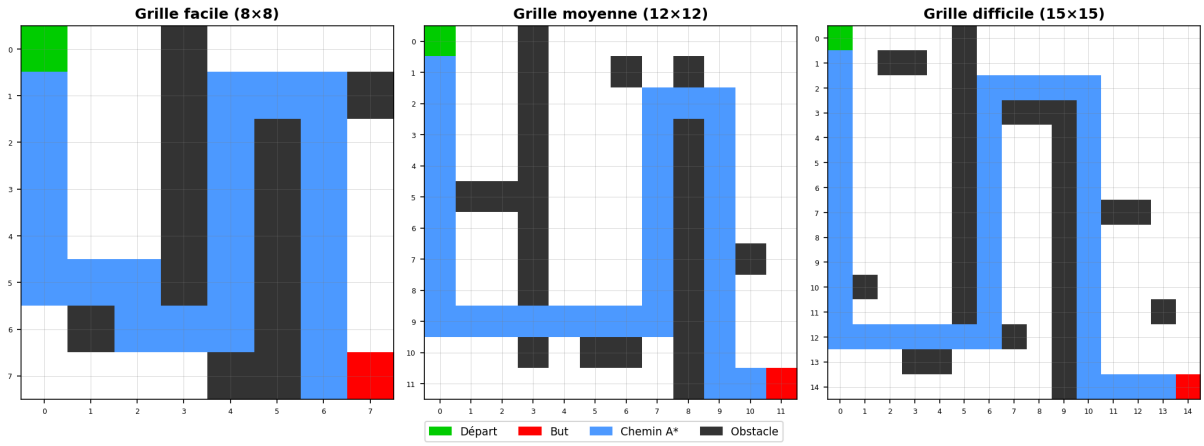

Figure 1 – Visualisation des trois grilles avec le chemin A^* optimal. Les cellules noires sont les obstacles, le vert indique le départ s_0, le rouge le but g.

4 Expériences et résultats

4.1 E.1 : Comparaison UCS vs Greedy vs A^*

Les trois algorithmes sont exécutés sur chaque grille. Le tableau 2 synthétise les résultats.

Table 2 – Résultats de E.1 : comparaison des algorithmes sur trois grilles.

Grille	Algorithme	Coût	Nœuds dév.	Max OPEN	Temps (ms)
Facile	UCS	24	49	4	0.53
	Greedy	26	33	14	0.41
	A^*	24	45	6	0.25
Moyenne	UCS	36	118	7	1.11
	Greedy	40	66	–	0.41 [1]
	A^*	36	104	10	0.77
Difficile	UCS	48	190	8	1.11
	Greedy	52	107	–	0.41[1]
	A^*	48	172	15	1.55

Analyse.

— **Optimalité.** UCS et A^* trouvent le chemin de coût minimal (24, 36, 48), confirmant l'optimalité avec une heuristique admissible. Greedy produit des chemins sous-optimaux (coûts 26, 40, 52), soit des surcoûts de 8 %, 11 % et 8 %.

— **Efficacité.** A^* développe moins de nœuds qu'UCS (45 vs 49, 104 vs 118, 172 vs 190), ce qui illustre l'élagage heuristique. UCS, sans information, explore la totalité des cellules accessibles.

— **Greedy.** Bien que Greedy développe le moins de nœuds (33, 66, 107), il sacrifie l'optimalité. La réduction du nombre de nœuds est trompeuse puisque le chemin résultant est plus coûteux.

La figure 2 superpose les chemins des trois algorithmes sur chaque grille, illustrant comment Greedy est piégé par les ouvertures asymétriques.

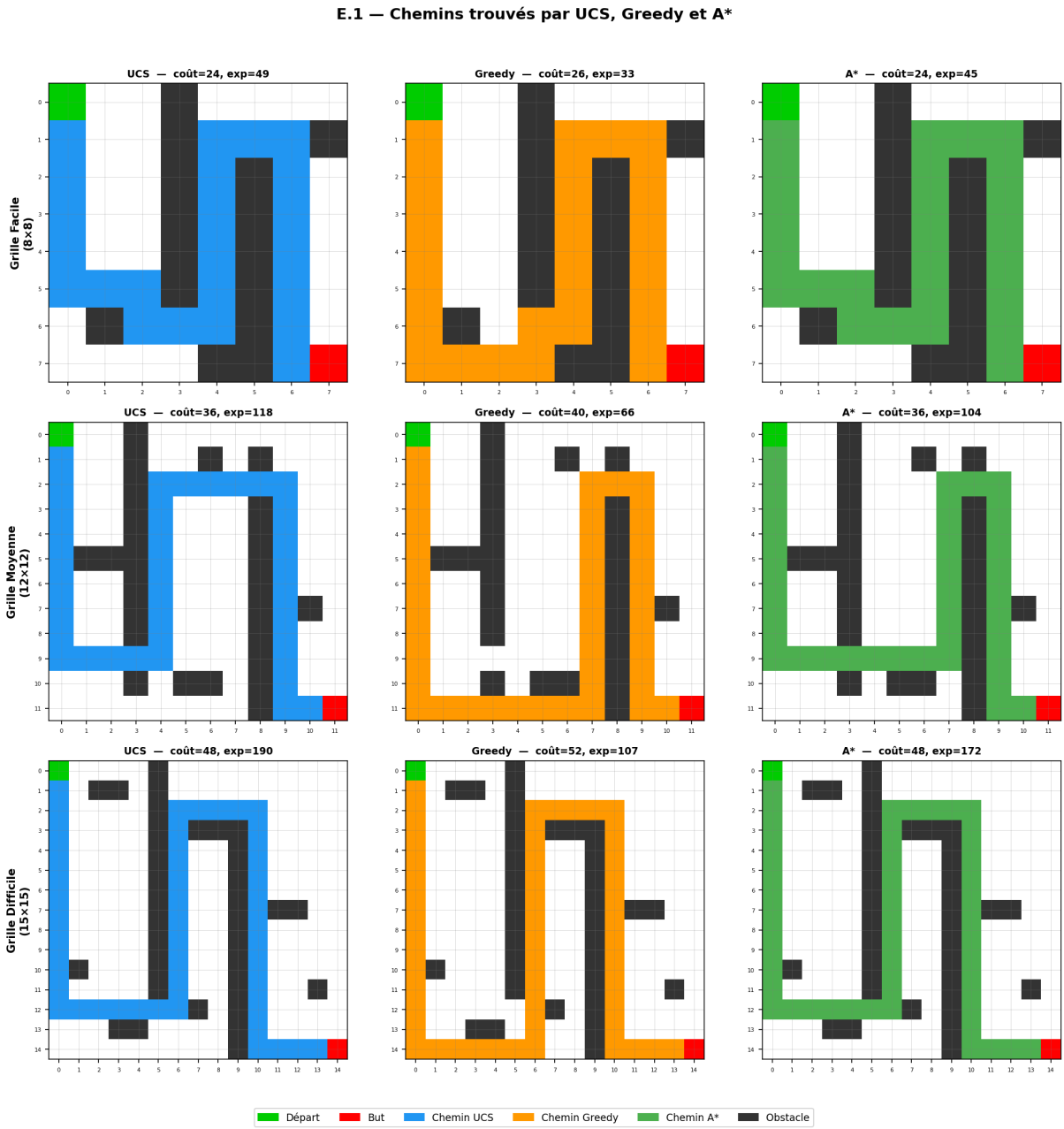

FIGURE 2 – Chemins trouvés par UCS, Greedy et A^* sur les trois grilles. Greedy emprunte un chemin sous-optimal sur chaque grille.

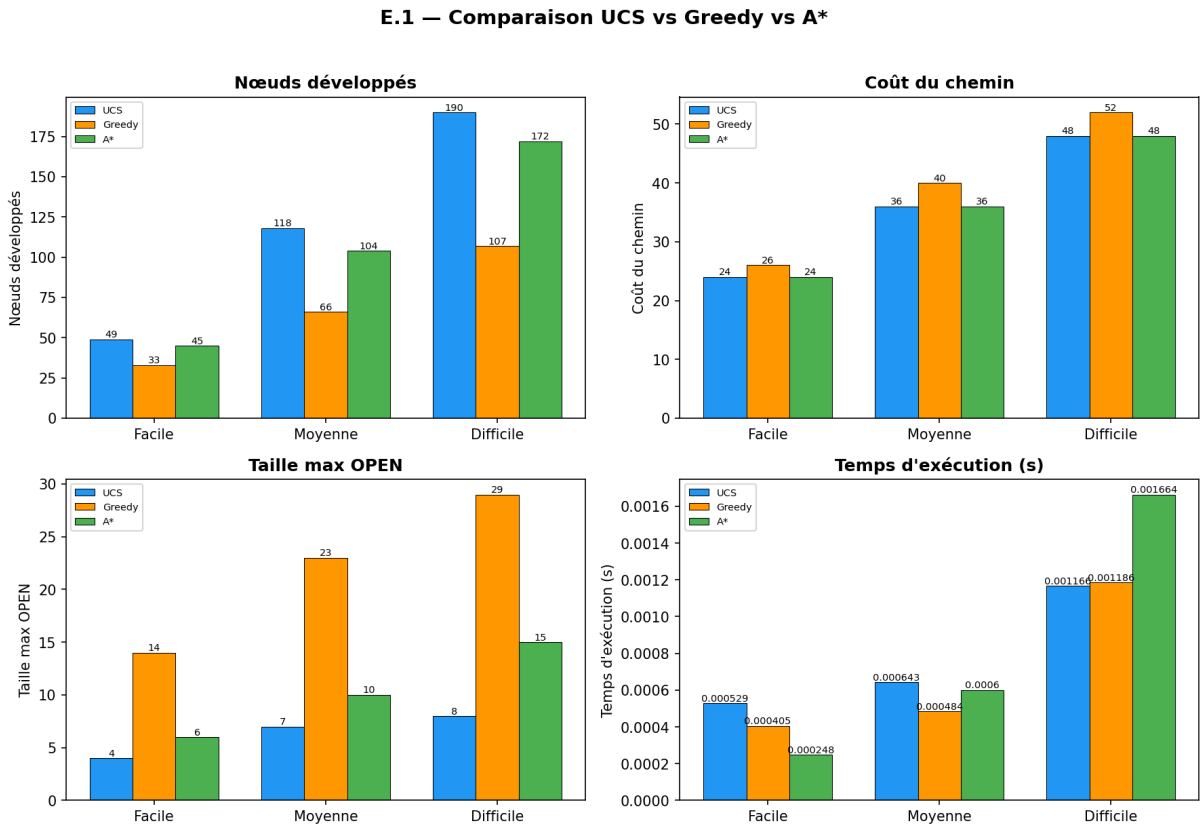

FIGURE 3 – Comparaison quantitative des trois algorithmes : nœuds développés, coût, taille OPEN et temps.

4.2 E.2 : Impact de l'incertitude ε sur la robustesse

On fixe le chemin A^* optimal et on varie $\varepsilon \in \{0, 0.1, 0.2, 0.3\}$. Pour chaque configuration, on calcule la probabilité d'absorption vers GOAL par la formule exacte (8) et par simulation Monte-Carlo ($N = 2000$ trajectoires).

TABLE 3 – Probabilité d'atteindre GOAL en fonction de ε : calcul analytique vs Monte-Carlo.

Grille		$\varepsilon = 0$	$\varepsilon = 0.1$	$\varepsilon = 0.2$	$\varepsilon = 0.3$
Facile	Analytique	1.0000	0.4149	0.1418	0.0372
	Monte-Carlo	1.0000	0.4300	0.1375	0.0320
Moyenne	Analytique	1.0000	0.1674	0.0202	0.0016
	Monte-Carlo	1.0000	0.1700	0.0160	0.0005
Difficile	Analytique	1.0000	0.0873	0.0049	0.0001
	Monte-Carlo	1.0000	0.0925	0.0035	0.0000

Analyse.

— Pour $\varepsilon = 0$ (déterministe), $\mathbb{P}(\text{GOAL}) = 1$ par construction.

— Dès $\varepsilon = 0.1$, la probabilité de succès chute drastiquement : de 1.0 à 0.41 (facile), 0.17 (moyenne) et 0.087 (difficile). La longueur du chemin amplifie la vulnérabilité :

sur un chemin de L pas, la probabilité de ne subir aucune déviation est environ $(1-\varepsilon)^L$.

— Pour la grille difficile à $\varepsilon = 0.3$, $\mathbb{P}(\text{GOAL}) \approx 10^{-4}$: le plan déterministe est quasi-inutilisable.

— L'accord entre calcul analytique et Monte-Carlo valide mutuellement les deux approches. Les écarts résiduels sont compatibles avec la variance d'échantillonnage pour $N = 2000$.

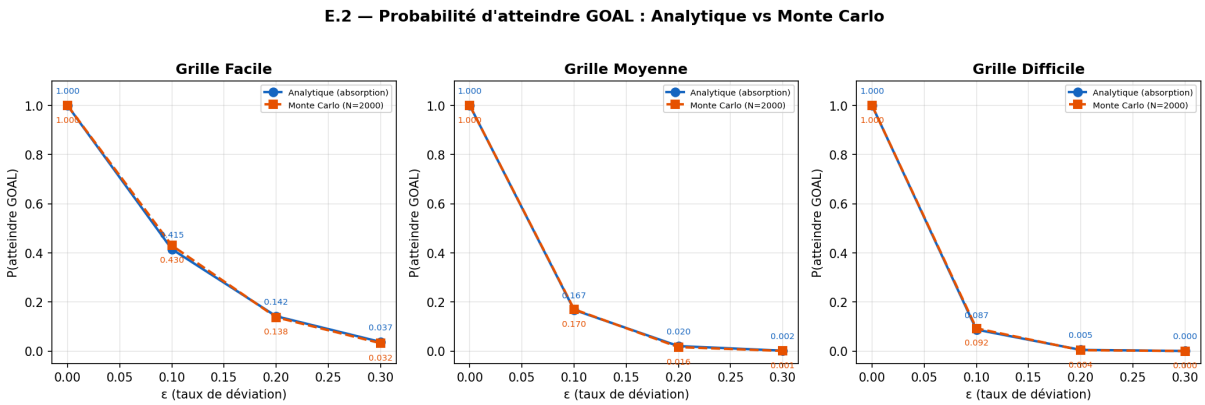

FIGURE 4 – $\mathbb{P}(\text{GOAL})$ en fonction de ε : analytique (traits pleins) vs Monte-Carlo (pointillés).

Évolution temporelle. La figure 5 montre l'évolution de $\pi^{(n)}[\text{GOAL}]$ pour différentes valeurs de ε. Lorsque $\varepsilon = 0$, la probabilité passe de 0 à 1 exactement au pas $n = L$ (le chemin est suivi déterministiquement). Pour $\varepsilon > 0$, la courbe croît progressivement et sature à une valeur strictement inférieure à 1, correspondant à la probabilité d'absorption $B_{s_0,\text{GOAL}}$.

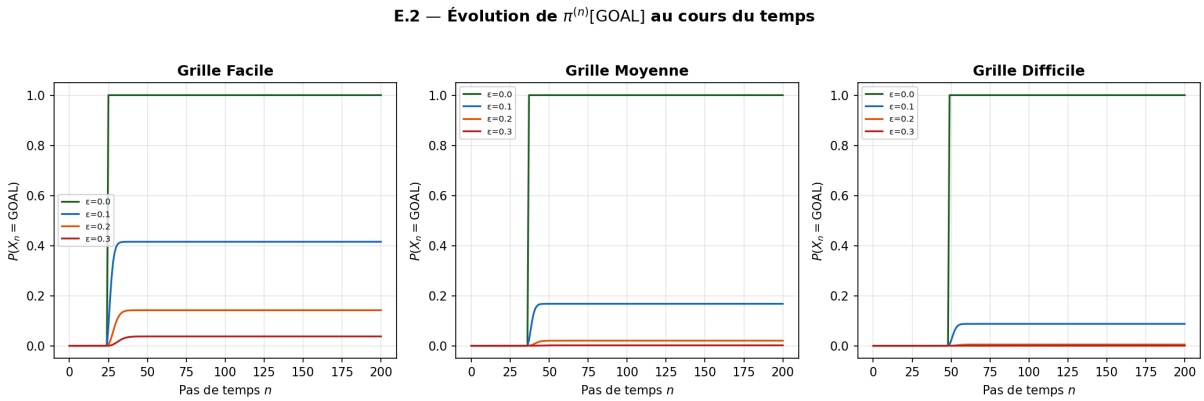

FIGURE 5 – Évolution de $\pi^{(n)}[\text{GOAL}]$ au cours du temps pour les trois grilles et quatre valeurs de ε.

Temps moyen d'absorption. Le tableau 4 présente le temps moyen avant absorption $\mathbb{E}[T_{\text{abs}}]$.

TABLE 4 – Temps moyen d'absorption $\mathbb{E}[T_{\text{abs}} \mid X_0 = s_0]$ en fonction de ε.

Grille	$\varepsilon = 0$	$\varepsilon = 0.1$	$\varepsilon = 0.2$	$\varepsilon = 0.3$
Facile	25.0	18.0	12.8	9.2
Moyenne	37.0	18.6	11.3	8.0
Difficile	49.0	19.9	11.1	7.8

Ce résultat peut sembler contre-intuitif : le temps d'absorption *diminue* lorsque ε augmente. L'explication est que l'absorption inclut FAIL : à ε élevé, l'agent quitte rapidement le chemin et est absorbé par FAIL en quelques pas, ce qui réduit le temps moyen global.

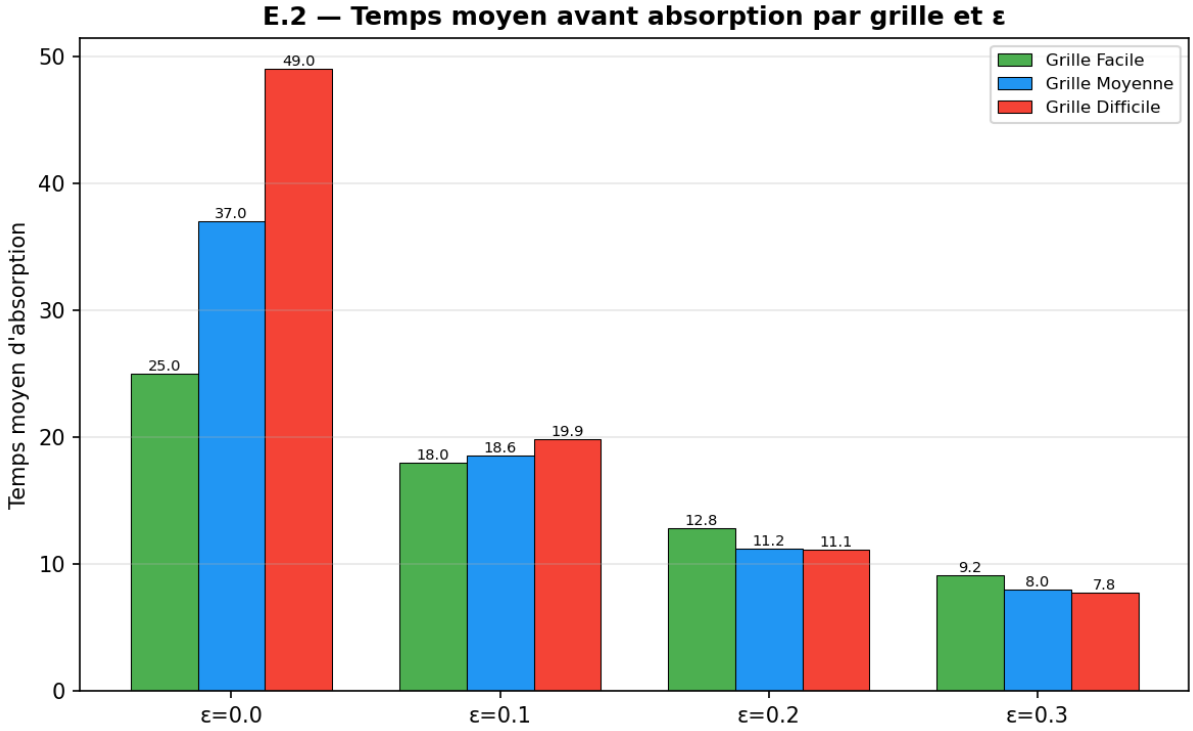

FIGURE 6 – Temps moyen d'absorption en fonction de ε pour les trois grilles.

4.3 E.3 : Comparaison de deux heuristiques admissibles

On compare A^* avec $h = 0$ (équivalent à UCS) et $h = h_{\text{Man}}$ (Manhattan). Les deux heuristiques sont admissibles ; Manhattan domine strictement $h = 0$.

TABLE 5 – Impact de l'heuristique sur le nombre de nœuds développés.

Grille	$h = 0$ (nœuds)	Manhattan (nœuds)	**Réduction**
Facile	49	45	$-8.2\,\%$
Moyenne	118	104	$-11.9\,\%$
Difficile	190	172	$-9.5\,\%$

Analyse. L'heuristique Manhattan réduit le nombre de nœuds développés de 8 à 12 % selon la grille, sans altérer l'optimalité du chemin (coût identique dans tous les cas). Ce

résultat illustre le théorème de dominance : si $h_2(n) \geq h_1(n)$ pour tout n, alors A^* avec h_2 développe au plus autant de nœuds qu'avec h_1, à condition que les deux soient admissibles et cohérentes [**?**].

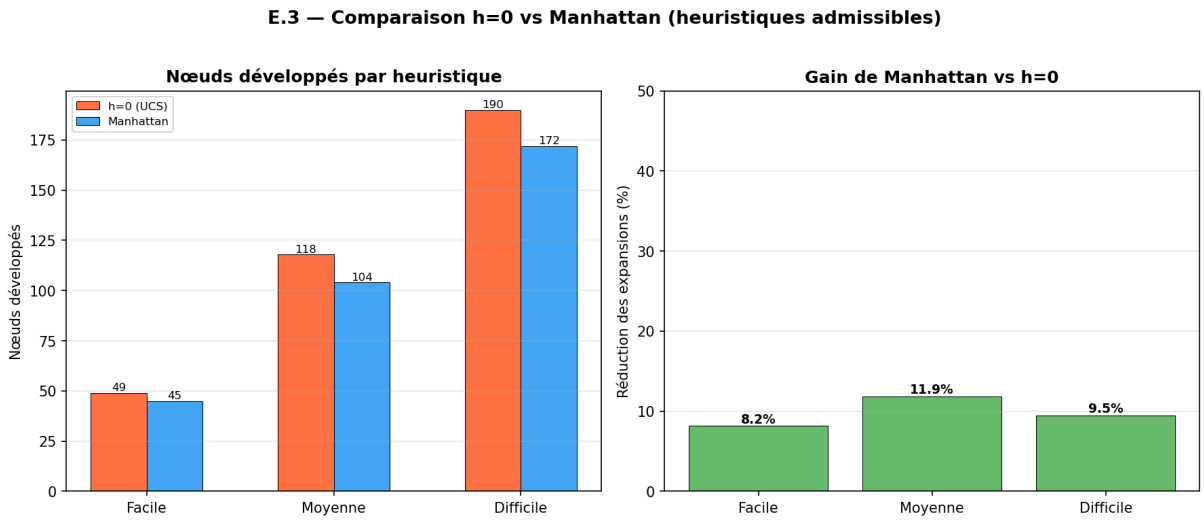

FIGURE 7 – Comparaison $h = 0$ vs Manhattan : nœuds développés et réduction relative.

4.4 E.4 : Weighted A^* : compromis vitesse vs optimalité

On fait varier $w \in \{1.0, 1.5, 2.0, 3.0, 5.0\}$ et on mesure le coût et les nœuds développés.

TABLE 6 – Résultats de Weighted A^* en fonction du poids w.

	Facile		**Moyenne**		**Difficile**	
w	Coût	Nœuds	Coût	Nœuds	Coût	Nœuds
1.0	24	45	36	104	48	172
1.5	24	40	36	84	48	127
2.0	26	40	36	84	52	127
3.0	26	40	40	81	52	127
5.0	26	38	40	73	52	123

Analyse.

- Pour $w = 1$ (A^* standard), le coût est optimal. À mesure que w augmente, le coût croît (par exemple, de 48 à 52 pour la grille difficile), restant borné par $w \cdot C^*$.
- Le nombre de nœuds développés décroît significativement : sur la grille difficile, de 172 ($w = 1$) à 123 ($w = 5$), soit une réduction de 28 %.
- La grille facile atteint rapidement un plateau : le surcoût apparaît dès $w = 2$ (coût 26 au lieu de 24), et le gain en expansions est marginal au-delà.
- Ce compromis est particulièrement utile dans des contextes temps-réel où une solution "suffisamment bonne" rapidement calculée est préférable à l'optimale.

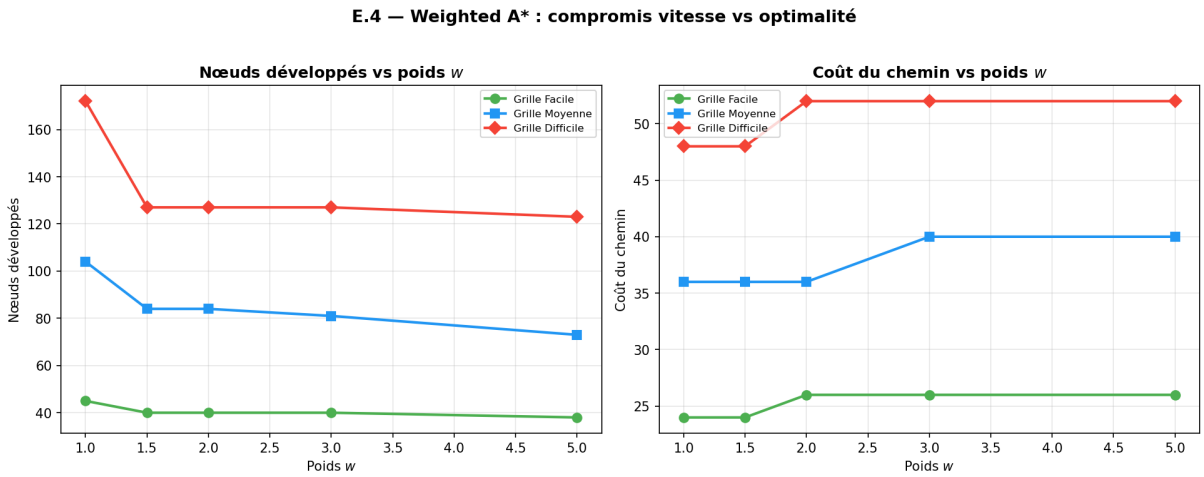

FIGURE 8 – Weighted A^* : nœuds développés et coût en fonction de w pour les trois grilles.

4.5 Validation Monte-Carlo

Pour compléter l'analyse, $N = 5000$ trajectoires sont simulées sur la grille difficile avec $\varepsilon = 0.2$. La figure 9 montre cinq trajectoires échantillons (la plupart échouent après quelques pas) et la distribution du temps d'atteinte pour les rares trajectoires atteignant GOAL.

Avec $\mathbb{P}(\text{GOAL}) \approx 0.005$, seules environ 25 trajectoires sur 5000 atteignent le but. Le temps moyen d'atteinte conditionnel (pour les trajectoires réussies) est de l'ordre de 54 pas, ce qui excède le coût déterministe de 48 : les trajectoires survivantes subissent des déviations latérales qui allongent le parcours avant de converger vers le but.

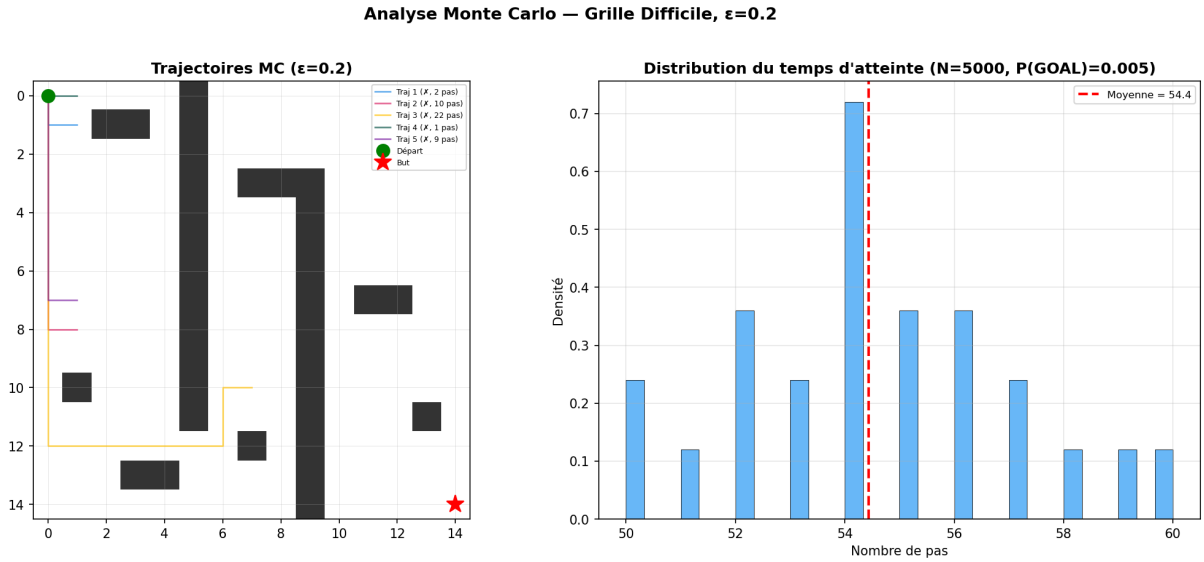

FIGURE 9 – Analyse Monte-Carlo sur la grille difficile ($\varepsilon = 0.2$) : trajectoires échantillons et distribution des temps d'atteinte.

5 Discussion

5.1 Intérêt et garanties des heuristiques

Les résultats confirment le rôle central de l'heuristique dans les algorithmes de recherche :

— Une heuristique admissible et cohérente (Manhattan) garantit l'optimalité du chemin trouvé par A^*.
— Le théorème de dominance se vérifie empiriquement : Manhattan ($h \geq 0$ partout) réduit les expansions de 8 à 12 % par rapport à $h = 0$.
— Greedy Best-First, bien qu'efficace en termes de nœuds, perd la garantie d'optimalité et produit des chemins 8 à 11 % plus coûteux.
— Weighted A^* offre un compromis paramétrable via w, avec une borne de sous-optimalité $w \cdot C^*$.

5.2 Impact de l'incertitude sur un plan déterministe

L'expérience E.2 met en évidence la fragilité d'un plan déterministe face à l'incertitude :

— La probabilité de succès décroît exponentiellement avec la longueur du chemin et le taux de déviation. Heuristiquement, $\mathbb{P}(\textsc{Goal}) \sim (1-\varepsilon)^L$ pour un chemin de longueur L sans rattrapages possibles.
— La politique restreinte au chemin (pas de re-routage) est particulièrement vulnérable : une seule déviation suffit à déclencher l'absorption en Fail.
— Les grilles plus complexes (chemin plus long) amplifient ce phénomène : à $\varepsilon = 0.1$, $\mathbb{P}(\textsc{Goal})$ passe de 0.41 (facile, $L = 25$) à 0.087 (difficile, $L = 49$).

5.3 Chemin optimal prévu vs performance probabiliste réelle

Le décalage entre optimalité déterministe et performance stochastique constitue l'enseignement principal du projet :

— Le chemin A^* de coût 48 (grille difficile) n'a que 8.7 % de chances d'être suivi avec succès à $\varepsilon = 0.1$.
— Les trajectoires Monte-Carlo réussies prennent en moyenne 51 pas (au lieu de 48), à cause des déviations latérales rattrapées.
— Un chemin plus court (au sens déterministe) n'est pas nécessairement le plus robuste : un chemin légèrement plus long mais passant par des "corridors larges" (plus de cellules voisines libres) offrirait potentiellement une meilleure $\mathbb{P}(\textsc{Goal})$.

6 Conclusion et perspectives

Ce projet a permis d'étudier l'articulation entre planification heuristique déterministe et analyse stochastique de l'exécution. Les principaux résultats sont :

1. A^* avec Manhattan est optimal et plus efficace qu'UCS (réduction de 8–12 % des nœuds développés), tandis que Greedy sacrifie l'optimalité au profit de la rapidité.
2. L'incertitude d'exécution ε dégrade rapidement la probabilité de succès d'un plan déterministe, en particulier sur les chemins longs. Pour la grille difficile, $\mathbb{P}(\textsc{Goal})$ chute de 1.0 à 0.087 dès $\varepsilon = 0.1$.

3. La concordance entre le calcul analytique (matrice fondamentale N) et les simulations Monte-Carlo valide la modélisation markovienne.
4. Weighted A^* offre un compromis explicite entre optimalité et coût computationnel, avec une réduction d'expansions pouvant atteindre 28 %.

Perspectives

Plusieurs pistes d'amélioration se dégagent :

— **Re-planification en ligne.** Étendre la politique à l'ensemble des états libres (pas uniquement au chemin) ou adopter une stratégie de re-planification (D^* Lite, LPA*) permettrait de corriger les déviations en temps réel.

— **Heuristiques apprises.** Des réseaux de neurones entraînés sur des instances similaires pourraient fournir des heuristiques plus informatives tout en restant admissibles, améliorant l'élagage de A^*.

— **Variantes mémoire.** Les algorithmes IDA^* (Iterative Deepening A^*) et SMA^* (Simplified Memory-Bounded A^*) offrent des compromis temps-mémoire intéressants pour les grands espaces d'états.

— **Planification robuste.** Intégrer directement le critère stochastique dans la recherche — par exemple, maximiser $\mathbb{P}(\textsc{Goal})$ plutôt que minimiser le coût — conduirait à des solutions intrinsèquement robustes (MDP, Value Iteration).

— **Coûts hétérogènes.** L'extension à des coûts de déplacement variables (terrain, énergie) et à des heuristiques plus sophistiquées (pattern databases, heuristiques landmark) constituerait un cadre expérimental plus riche.